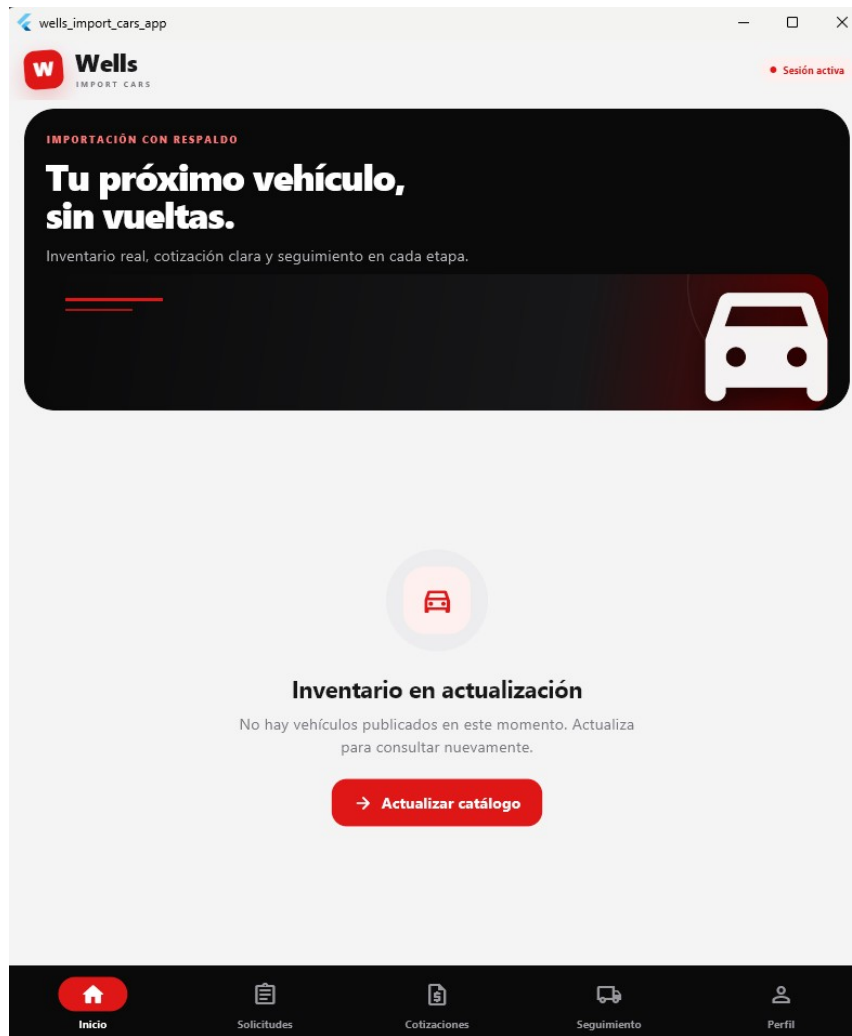


Pantalla de inicio y catalogo



Esta es la pantalla principal de la aplicación. Desde aquí el cliente puede revisar el catálogo de vehículos y moverse entre Inicio, Solicitudes, Cotizaciones, Seguimiento y Perfil. En la evidencia no hay vehículos publicados, por eso se muestra el estado vacío y el botón para actualizar el catálogo.

Código relacionado (simplificado)

```
Future<void> cargarCatalogo() async {  
  setState(() => cargando = true);  
  vehiculos = await CatalogoService().obtenerCatalogo();  
  setState(() => cargando = false);  
}  
  
if (vehiculos.isEmpty) mostrarCatalogoVacio();
```

Funcionamiento

La pantalla consulta la API mediante un servicio. Mientras espera puede mostrar un indicador de carga; si la lista llega vacía muestra el mensaje de inventario en actualización. El botón Actualizar catálogo vuelve a ejecutar la consulta.

Solicitud de vehiculo

well's_import_cars_app

← Solicitar vehículo

Dinos qué vehículo buscas.
Nuestro equipo usará estos criterios para preparar opciones reales.

PREFERENCIAS

Vehículo ideal

Marca deseada: toyota

Modelo deseado: hilux

Año deseado: 2020

Presupuesto máximo: 35000

Tipo de carrocería: pick up

Tipo de combustible: diesel

DETALLES ADICIONALES

Observaciones

Información opcional

➤ Enviar solicitud

En esta pantalla el cliente registra las características del vehículo que desea buscar. Se solicitan datos como marca, modelo, año, presupuesto máximo, carrocería, combustible y observaciones. Esta información se envía a la API para crear una nueva solicitud.

Código relacionado (simplificado)

```
final solicitud = {  
  'marcaDeseada': marcaController.text,  
  'modeloDeseado': modeloController.text,  
  'anioDeseado': int.parse(anioController.text),  
  'presupuestoMax': double.parse(presupuestoController.text),  
  'observaciones': observacionesController.text,  
};  
await SolicitudService().crearSolicitud(solicitud);
```

Funcionamiento

Al presionar Enviar solicitud se validan los campos y se ejecuta una operación POST contra la API. Mientras se envía, el botón puede deshabilitarse y mostrarse un estado de carga. Si la API responde correctamente se confirma que la solicitud fue registrada.

Solicitud de cotizacion

Esta pantalla permite pedir una cotizacion relacionada con una solicitud existente. El cliente selecciona la referencia de la solicitud y puede escribir una observacion para explicar que desea conocer. El empleado recibe la peticion y posteriormente genera la cotizacion final.

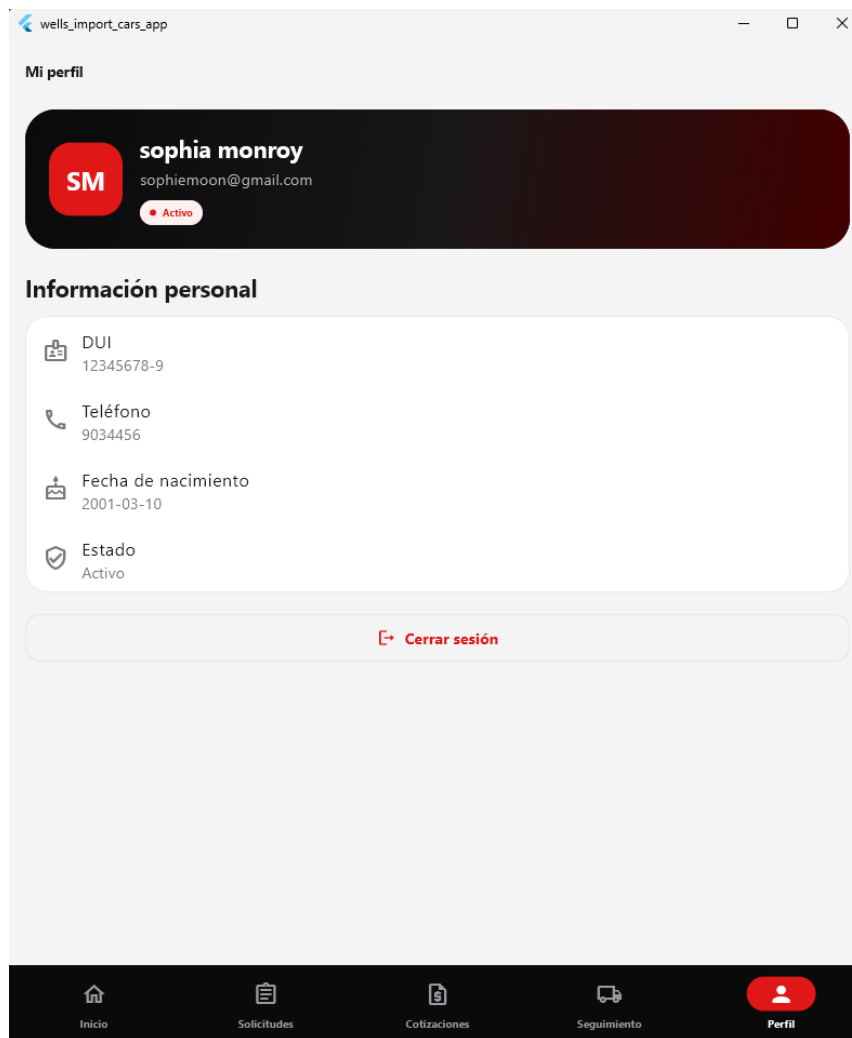
Codigo relacionado (simplificado)

```
final datos = {  
  'idSolicitud': solicitudSeleccionada.idSolicitud,  
  'observaciones': observacionesController.text,  
};  
  
await CotizacionService().solicitarCotizacion(datos);
```

Funcionamiento

La aplicacion carga las solicitudes del cliente y permite escoger una. Luego envia la peticion de cotizacion a la API. La pantalla diferencia la solicitud del cliente de la cotizacion final, ya que los precios y conceptos son preparados por el personal de Wells Import Cars.

Perfil del cliente



La pantalla de perfil muestra la información básica del cliente autenticado: nombre, correo, DUI, teléfono, fecha de nacimiento y estado de la cuenta. También incluye la opción para cerrar la sesión.

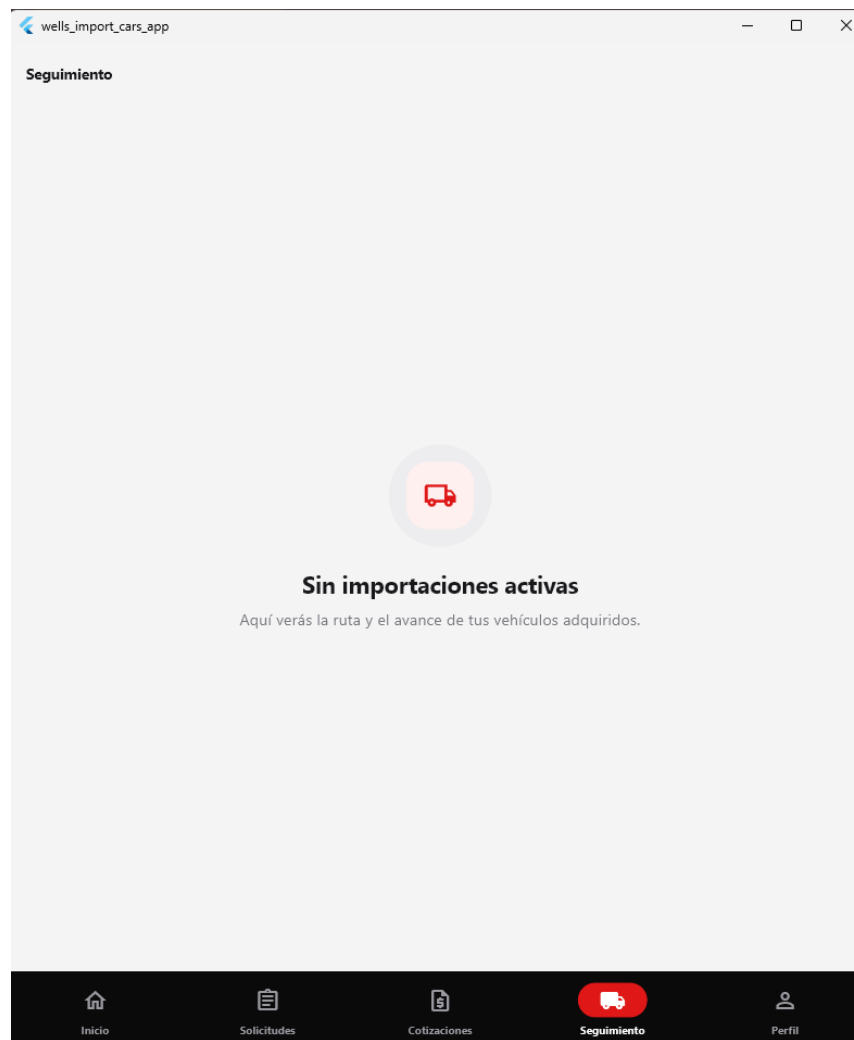
Código relacionado (simplificado)

```
final usuario = SessionService.usuarioActual;  
  
Text(usuario.nombre);  
Text(usuario.correo);  
Text(usuario.dui);  
  
SessionService.cerrarSesion();
```

Funcionamiento

Los datos mostrados corresponden al usuario de la sesión activa. Al cerrar sesión se elimina el token guardado en memoria y se regresa a la pantalla de inicio de sesión, evitando que el usuario siga entrando a funciones privadas.

Seguimiento de importaciones



En Seguimiento el cliente puede consultar el avance de los vehiculos asociados a sus procesos de importacion. En esta evidencia no existen importaciones activas, por eso la interfaz muestra un estado vacio en lugar de una lista.

Codigo relacionado (simplificado)

```
Future<void> cargarSeguimiento() async {  
  setState(() => cargando = true);  
  importaciones = await SeguimientoService().obtenerSeguimiento();  
  setState(() => cargando = false);  
}  
  
if (importaciones.isEmpty) mostrarSinImportaciones();
```

Funcionamiento

La pantalla consulta la API y muestra los procesos que correspondan al cliente. Cuando existan registros se pueden presentar estados como comprado, en transito, en aduana, en taller o entregado. Si la API no devuelve datos, se muestra el mensaje Sin importaciones activas.